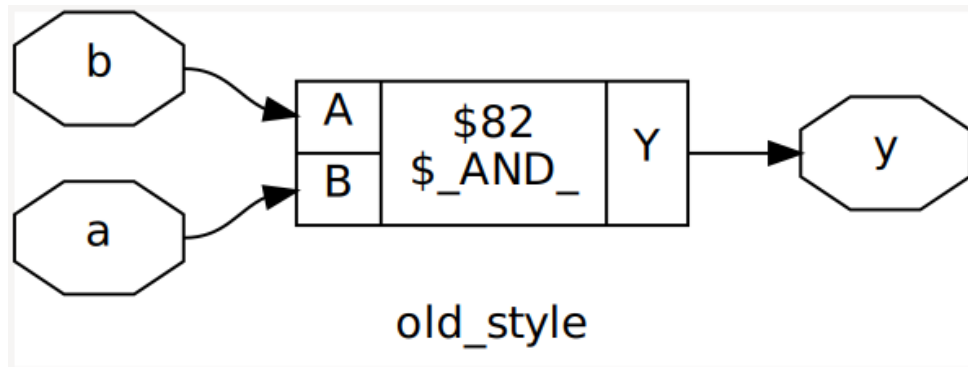


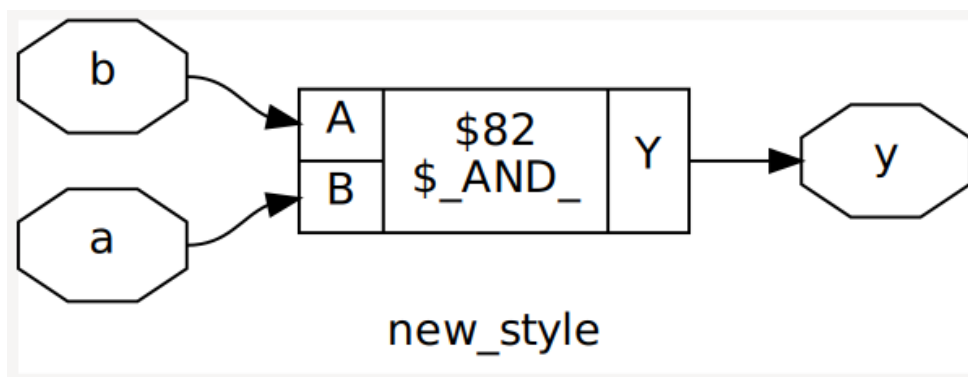
Day 13

System Verilog:

```
module old_style(  
  input wire a, b,  
  output reg y  
);  
always @(*) y = a & b;  
endmodule
```



```
module new_style(  
  input logic a,b,  
  output logic y  
);  
always_comb y = a&b;  
endmodule
```



Both provides the same hardware, but System Verilog reduces the complexity of choosing between reg and wire.

Introduction to Linting:

Linting is a **static code analysis** process used to check Hardware Description Language (HDL) code—like Verilog, VHDL, or SystemVerilog—for potential errors, design violations, and stylistic issues without actually running a simulation.

```
module latching_sv(  
  input sel, a, b,  
  output logic y);  
  
always_comb begin  
  if(sel) y = b;  
end  
endmodule
```

Here, a is left latching , hence after linting we can see the warning

Command: iverilog -g2012 -Wall -o latching_sv latching_sv.sv

%Warning-UNUSED SIGNAL: latching_sv.sv:2:13: Signal is not used: 'a'
: ... note: In instance 'latching_sv'

```
2 | input sel, a, b,
   |      ^
```

... For warning description see <https://verilator.org/warn/UNUSED SIGNAL?v=5.020>

... Use `"/* verilator lint_off UNUSED SIGNAL */"` and `lint_on` around source to disable this message.

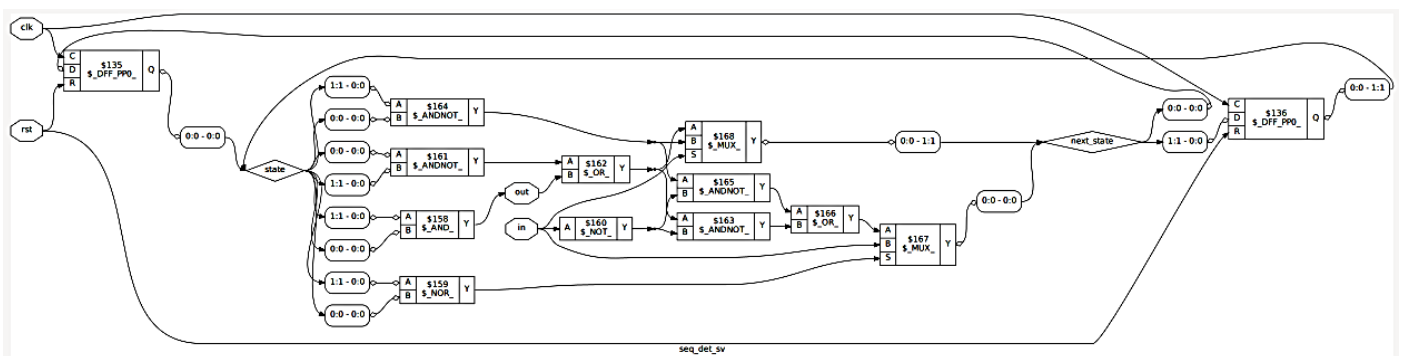
%Warning-LATCH: latching_sv.sv:5:2: Latch inferred for signal 'y' (not all control paths of combinational always assign a value)

: ... Suggest use of `always_latch` for intentional latches

```
5 | always_comb begin
   | ^~~~~~
```

%Error: Exiting due to 2 warning(s)

Sequence Detector using System Verilog



4. Printing statistics.

=== seq_det_sv ===

Number of wires:	14
Number of wire bits:	16
Number of public wires:	6
Number of public wire bits:	8
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	13
\$_ANDNOT_	4
\$_AND_	1
\$_DFF_PPO_	2
\$_MUX_	2
\$_NOR_	1
\$_NOT_	1
\$_OR_	2